Lecture 15 · Part IV · Lecture notes

Time series

Dataset: Chicago transit ridership

How to use these notes

The primary text for this lecture is Géron, Chapter 13. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A new kind of data

Everything so far assumed the rows were **exchangeable**: a random split was valid because any row was as good as any other for training.

Not here. In a time series the rows are ordered and neighbouring rows are alike — one day's ridership tells you most of the next day's. Split such data at random and a point's own near-future lands in the training set, so the score you report is not the score you would get in deployment. It is optimistic by a margin we will measure.

1.1 The brief

Forecast the number of people who will board rail and bus tomorrow, from twenty years of daily boarding totals. The decisions it changes — how many train sets leave the depot, how many drivers are rostered, whether a spare crew is held — are made the evening before, which fixes the horizon at one day and fixes when the forecast must exist: *before the day it describes*.

The two errors are not equal, and we choose a symmetric metric anyway

Forecast too high and empty vehicles run: fuel, crew hours, a line in the budget. Measurable and survivable. Forecast too low and passengers are left on platforms at peak: the cost is political and appears in no table you will be shown.
We say that out loud now rather than discovering it later.

The label is free — nobody annotates anything, because the next day's value is the label for today's window. But the rows are not exchangeable: row 400 is not a fresh draw from the same distribution as row 10, it is the same city eighteen months later, and rows 400 and 401 are almost the same measurement made twice.

1.2 Four lines of tidying, one of which matters

The file ships with 7,701 rows and 7,639 distinct ones: sixty-two days appear twice, identically, and nothing warns you. Reviewer question 4 from Lecture 1 — what was dropped? — and here the answer is 62, which you only know because you counted.

One categorical column, `day_type`: 5,336 ordinary working days, 1,216 marked U (the second weekend day and public holidays) and 1,087 marked A. This column is unusual and worth pausing on: *we know it in advance*. The next day's type is on the calendar today, so it is a legitimate input to a forecast of that day. Most features are not like that.

Plotting twenty-one years shows three things, none of which is what we will model: bus falling while rail rose and then fell, an annual wobble of about a tenth of the level, and March 2020 as a

cliff rather than a dip. A model fitted across that cliff is being asked to describe two different cities, so we model 2016 to 31 May 2019 — 1,247 days of one regime. Everything after is set aside, not because it is uninteresting but because we cannot both use it and claim to have been surprised by it. Data snooping applies to time as well as to rows.

Zooming in until individual days are visible shows a seven-day cycle so strong it dominates the plot, and one trough in the wrong place. Ask the data rather than the calendar: the three days around it are marked A, U, U — a public holiday extending the weekend, which the column we were given already knows. *Every anomaly should end either in an explanation or in a note that you could not find one. Not in silence.*

2 Derivation: stationarity, differencing and autocorrelation

Every model in this course assumes the data it was trained on and the data it will meet are the same kind of thing. For a series, what does that assumption say — and does ours satisfy it?

Definition 2.1 (Weak stationarity)**.** $\{y_t\}$ *is weakly stationary if* $\mathbb{E}[y_t] = \mu$ *is constant,* $\mathrm{Var}(y_t) = \sigma^2$ *is constant and finite, and* $\mathrm{Cov}(y_t, y_{t+k}) = \gamma_k$ *depends on* k *only.*

Only the first two moments, and only the requirement that the covariance *forgets when and remembers only how far apart*. Strict stationarity — the whole joint law invariant under a shift — is far more than we need and more than any real series has.

Why a model depends on it

Fitting minimises an average loss over the training rows, and we then quote that as an estimate of the loss on future rows. That step is a *claim*: the future rows come from the same distribution. For rows indexed by time, "the same distribution" is exactly what weak stationarity asserts about the first two moments.
If μ drifts, a model fitted on the past is systematically wrong on the future — not noisily wrong, *biased*. That is the difference between an error that averages out and one that accumulates.

Our rolling mean moves by more than 170,000 riders across the record, so the first condition fails outright. An augmented Dickey–Fuller test returns -4.61 with $p \approx 0.0001$ and rejects its null — but read what was rejected: the null is the presence of a *unit root*, one specific way of being non-stationary. Rejecting it does not establish stationarity, which the plot already refuted. A test answers the question it was given, and ours was not that question.

2.1 Differencing

With $(\nabla y)_t = y_t - y_{t-1}$, applied to a sampled function ∇ is a discrete derivative. One value is lost at the start each time; count those losses, because `diff` returns `NaN` there and says nothing.

∇^d annihilates a polynomial trend of degree d

Monomials suffice, as ∇ is linear:

$$\begin{aligned} t^d - (t-1)^d &= \sum_{j=0}^{d-1} \binom{d}{j} (-1)^{d-1-j} t^j \\ &= d\, t^{d-1} + \text{lower order.} \end{aligned}$$

The leading coefficient is $d \neq 0$, so the degree drops by *exactly* one. Induct.

A linear series becomes a constant, and the constant is the slope — and a constant mean is the first condition of weak stationarity, which we have just bought. That d is the **order of integration**, the I in ARIMA.

Seasonal differencing, $(\nabla_s y)_t = y_t - y_{t-s}$, removes a periodic component *exactly*: if $y_t = c_t + r_t$ with $c_t = c_{t-s}$, then $(\nabla_s y)_t = r_t - r_{t-s}$, with no approximation and no fitted parameter.

Series	Standard deviation
the level	183,841
differenced once	198,098
differenced at lag 7	104,730

One difference *increased* the variability by 8%. That is not a bug and not an accident — there is an exact condition.

2.2 Autocorrelation, and when differencing helps

For a weakly stationary series, $\rho_k = \gamma_k/\gamma_0 \in [-1, 1]$ is the correlation of the series with itself k steps later. Note it is defined *only* under stationarity: without it the covariance depends on t as well as k and there is no single number to plot.

The variance of a lag-k difference

$$\begin{aligned} \operatorname{Var}(y_t - y_{t-k}) &= \operatorname{Var}(y_t) + \operatorname{Var}(y_{t-k}) - 2\operatorname{Cov}(y_t, y_{t-k}) \\ &= 2\sigma^2\big(1 - \rho_k\big). \end{aligned}$$

So differencing at lag k reduces the variance if and only if $\rho_k > \frac{1}{2}$.

Lag k	ρ_k	Predicted sd	Measured
1	0.419	198,236	198,098
7	0.831	106,782	104,730

The lag-1 prediction is right to one part in a thousand and the lag-7 to two per cent, the residual discrepancy being the series failing to be exactly stationary — which we already knew. And $\rho_1 < \frac{1}{2}$ while $\rho_7 > \frac{1}{2}$, so the first difference *must* be worse and the seventh *must* be better. Both were, before we knew why.

And now the naive forecast is explained

"Copy the value from seven days ago" has error $\hat{y}_t - y_t = -(\nabla_7 y)_t$ — its error *is* the seasonal difference. So its error variance is $2\sigma^2(1 - \rho_7)$, which at $\rho_7 = 0.831$ is $0.34\,\sigma^2$.
The naive forecast is hard to beat for one reason and one reason only: ρ_7 is large. It is not a lucky heuristic — it is the autocorrelation function, read off at a single lag.

One caution before converting that into an MAE

A Gaussian of standard deviation 104,730 would have mean absolute value $\sqrt{2/\pi}\,\sigma \approx 83{,}562$. Our measured MAE is 55,051.
The errors are not Gaussian: the excess kurtosis of the seven-day difference is 12 — most days are almost exactly like the same day last week, and a handful are catastrophically different. A variance is the wrong summary of that distribution, and so is any conversion assuming normality.
Which is also the argument for MAE over RMSE, arriving from a second direction.

3 Baselines, and the metric

We predict $\hat{y}_{t+1} = f(y_{t-55}, \ldots, y_t)$: given the last eight weeks, the next day. MAE is the choice, because the units are *passengers* and the operations team can act on "we were out by forty thousand boardings"; because the worst days are holidays the calendar already tells us about, and we do not want the metric dominated by days you can look up; and because RMSE would make one bad holiday worth more than a whole ordinary month. MAPE is reported beside it, never as the objective — it divides by the truth, so it is most sensitive exactly where the model is weakest, and a metric like that makes every improvement look like noise.

Three baselines, all scored on exactly the rows the models will be scored on: predict a constant, copy the day before, copy last week.

The number to beat

Copying the value from seven days ago gives an MAE of **55,399** over 1,191 days — 12.0% of a typical day, and 2.4 times better than copying the previous day. The seven-day cycle is worth more than recency.
It needs no training, no maintenance, no monitoring and no data scientist, and it fails only on days that appear on a public calendar. A transit authority could put it into service in an afternoon.

One holiday costs it 464,640 in a single day — more than eight ordinary days of error put together. Any model has to be better than this *by enough to justify existing*, and you should write down what "enough" is before you see any model output.

3.1 Turning one series into a table

The dataset class is nine lines, and the subtlety is entirely in `__len__` and in the single index that is both the end of the window and the position of the label. Test it on a series whose answer you know: a toy of $0, 1, 2, 3, 4, 5$ with window 3 gives $[0, 1, 2] \to 3$, $[1, 2, 3] \to 4$, $[2, 3, 4] \to 5$ —

three rows, three targets, none inside its own window. Twenty seconds, and it is the only thing standing between you and an off-by-one that produces a beautiful score.

Eight weeks is a whole number of seven-day cycles, so every position sees the same phase of the week; long enough to contain a slow change in level; and short enough that 1,247 days still yield 1,191 rows. Longer windows cost rows — a 365-day window gives 882 instead of 1,191, so the model has more to learn from less.

Dividing by a million puts every value near $[0, 1]$ where the default initialisation and learning rate expect their inputs. That is scaling, and unlike `StandardScaler` it learns nothing from the data, so it cannot leak.

4 The protocol, and what it is worth

$\rho_7 = 0.831$ says any two days a week apart are strongly dependent. Cross-validation assumes the held-out rows are independent of the rows the model was fitted on. *Those two sentences cannot both be acted on.*

Read our own reproducibility rule again

Lecture 2's checklist says: "Seeds for every generator; `KFold(shuffle=True, random_state=42)`, never the default."
That rule was correct, and it solved a real problem: a frame sorted differently gave different folds and nobody could see why. *Here it is the bug.*
A rule you cannot say the reason for is a rule you will apply where it does not hold. This is the clearest example the course contains.

Cross-validation is unbiased for the risk under two conditions: the held-out rows are independent of the training rows, and they are drawn from the distribution the model will meet. A shuffled split of a time series satisfies neither.

Condition 1 fails and ρ_k says by how much: the covariance is $0.83\sigma^2$ at lag 7 and never near zero for any lag inside the window. A model fitted on rows *surrounding* a held-out row faces a smaller conditional variance than one fitted only on rows before it, so the estimate is biased downwards — optimistic in the overwhelming majority of cases. Not "always": that would be a theorem, and the conditional-variance argument is a heuristic rather than a bound.

Condition 2 fails and it is not subtle. In deployment every training day precedes every day being forecast; under a shuffled split, four training days in five come *after* the day being scored. The model is not forecasting. It is being handed the shape of the series around the point it is asked about, and interpolating — a different and much easier problem, and nobody is paying for it.

Model	Random split	Forecasting forward	Gap
Linear, 56 lags	44,925	56,446	+25.6%
Simple RNN, 32 units	38,224	49,073	+28.4%

4.1 Now do the thing this course exists for

We have a gap of 28%. *How much of it is the split?* Three other things changed when the protocol changed, and each moves the number on its own.

Confound 1 — the test rows are harder. Copying last week needs no fitting, so its error measures only how difficult a set of days is. On the shuffled folds it scores 55,385; on the five months after the cut, 64,815. The forecast period is 17% harder before any model is involved, because it contains the winter holidays. This is Lecture 2's "we dropped the capped districts" returning: any time a number moves, ask first whether the evaluation set moved.

Confound 2 — the training set changed size. `TimeSeriesSplit`'s first fold fits on a fifth of the rows and scores 80,506, which is not a statement about the protocol but about having 238 training rows. The repair is a rolling-origin backtest with a *fixed* training window — 700 rows for every origin, five origins, 98 days scored each time — so the folds differ only in *when* they are.

Confound 3 — the folds have a spread. The random folds span 39,538 to 51,096. A difference in means smaller than that spread is not a difference: the paired-comparison lesson from Lecture 2, unchanged.

Protocol	Model	Naive	Ratio
random draws of 98 rows, 20 of them	48,093	61,461	0.782
rolling origin, 5 origins	47,212	55,371	0.853

Same training size, same test size, same model — only the arrangement differs — and each scored against copying last week *on its own rows*, which cancels the difficulty of the period. The random split reports the model as 9.0% more skilful than it is.

The honest verdict, in three numbers

28% is the raw gap on the number you committed. That is what you would have been wrong by, and it is real.
17% of it is the forecast period genuinely being harder — the baseline moves too.
9 to 10% is the protocol itself, and it survives every control we could apply: matched training size, matched test size, and the baseline computed on identical rows. The linear model is overstated by 7.4% and the recurrent network by 9.7%, which is what you expect if the cause is the split rather than the model.

Put in the terms a stakeholder hears: we told them the recurrent model was 31% better than doing nothing, and it is 24% better. A fifth of the case for building it was an artefact of the evaluation, and our own linear model loses a third the same way, 18.9% to 12.9%.

	Scaling before the split	Random split on a series
Diagnosed in	Lecture 2	here
Cost, measured	nothing a measurement could find	a fifth to a third of everything a model was worth
Why that size	an invertible affine map on an equivariant estimator	ρ_k large at every lag inside the window

Same procedural rule, two wildly different costs. *You cannot tell which case you are in from the number — only from the structure. That is why the rule is procedural.*

5 The linear model

$$\hat{y}_{t+1} = b + \sum_{k=0}^{55} w_k \, y_{t-k}.$$

Fifty-six weights and a bias — and if the model learns $w_6 = 1$ with every other weight zero, it has reinvented copying last week. *The linear model contains the baseline*, so it cannot do worse unless we fit it badly.

It scores 44,925 against 55,399: 18.9% better, with folds from 39,538 to 51,096, and over twenty different shuffles the mean moves by about 300 — so the improvement is far larger than the noise in the estimate.

What did the weights learn? The largest is at lag 1 at +0.45, so the day before still carries the level. After that the next four largest are at lags 35, 7, 14 and 28 — all seven multiples of seven have a positive weight and no other lag does so consistently. The weights sum to 0.86 with an intercept around 90,000 riders, so the model shrinks gently towards the mean.

It rediscovered the seven-day cycle from the data. Nobody told it that a week has seven days.

6 The notebook

What to change

Shuffle the rows before splitting and re-run the evaluation. The score improves, and nothing warns you. That improvement is the whole argument of the second half of this lecture, produced by you rather than quoted at you.

7 Exercises

1. State the three conditions of weak stationarity, and explain why a model that quotes a training average as an estimate of future loss depends on them. [5]

2. Prove that ∇ reduces the degree of a polynomial by exactly one, and state what ∇^d therefore does to a degree-d trend. [4]

3. Derive $\mathrm{Var}(y_t - y_{t-k})$ in terms of ρ_k, and give the exact condition under which differencing at lag k reduces the variance. [5]

4. A colleague evaluates a forecasting model with `KFold(shuffle=True)` and reports a 31% improvement over a naive baseline. Name the two conditions the protocol violates, and say in which direction the estimate is biased. [4]

5. A model scored under a forward protocol is 28% worse than under a random split. Name

three things other than the protocol that could explain part of that gap, and how you would control for each. [4]

Summary

Weak stationarity asks for a constant mean, a constant finite variance and a covariance depending only on the lag — and models depend on it because they quote a training average as an estimate of a future one. Our series does not have it. ∇^d annihilates a polynomial trend of degree d because ∇ drops the degree by exactly one, and ∇_s annihilates a component of period s exactly.

$\mathrm{Var}(y_t - y_{t-k}) = 2\sigma^2(1 - \rho_k)$, so differencing helps if and only if $\rho_k > \frac{1}{2}$ — which predicted, before it was measured, that the first difference would be worse and the seventh better. The naive forecast's error *is* the seasonal difference, so its strength is $\rho_7 = 0.831$ and nothing else.

Then the lecture's real subject. Lecture 2's shuffled-fold rule, correct where it was written, is the bug here: it violates both conditions cross-validation needs, and it reported the recurrent model as 31% better than nothing when it is 24% better. Of the raw 28% gap, 17% is the forecast period genuinely being harder and 9 to 10% is the protocol, surviving matched training size, matched test size and a baseline recomputed on identical rows.